This creates a new parameter pack `args`, which is a list of the function parameters rather than a list of types, which you can expand with `...` as before. Now, you can use a pattern with the pack expansion for declaring the function parameters, just as you can use a pattern when you expand the pack elsewhere. For example, this is used by the `std::thread` constructor to take all the function arguments by rvalue reference (see section A.1):

```
template<typename CallableType,typename ... Args>
thread::thread(CallableType&& func,Args&& ... args);
```

The function parameter pack can then be used to call another function, by specifying the pack expansion in the argument list of the called function. Just as with the type expansions, you can use a pattern for each expression in the resulting argument list. For example, one common idiom with rvalue references is to use `std::forward<>` to preserve the rvalue-ness of the supplied function arguments:

```
template<typename ... ArgTypes>
void bar(ArgTypes&& ... args)
{
    foo(std::forward<ArgTypes>(args)...);
}
```

Note that in this case, the pack expansion contains both the type pack `ArgTypes` and the function parameter pack `args`, and the ellipsis follows the whole expression. If you call `bar` like this,

```
int i;
bar(i,3.141,std::string("hello "));
```

then the expansion becomes

```
template<>
void bar<int&,double,std::string>(
    int& args_1,
    double&& args_2,
    std::string&& args_3)
{
    foo(std::forward<int&>(args_1),
        std::forward<double>(args_2),
        std::forward<std::string>(args_3));
}
```